

2你現在要繼續開發獨立專案：

FrameLab

這一輪不要擴張成更多功能，也不要急著整合所有大型 AI 模型。

這一輪的唯一目標是完成：

FrameLab V0.1 Core Loop

也就是把：

影片匯入 → 拆幀 → Timeline → Onion Skin → Frame/Region 選取 → 疊加式 AI 對話 → Context Engine → MCP → AI 分析

完整打通。

這一輪完成後，我希望可以真的在 FrameLab 裡做到：

「我點到某一幀，圈選角色的手，直接問 AI「這裡為什麼怪怪的？」」

AI 必須知道：

- 我目前在哪一個 Project
- 哪一條 Timeline
- 哪一個 Frame
- 選了哪些 Frames
- 選了哪個角色
- 圈了畫面的哪個 Region
- 前後幀是什麼
- Onion Skin 設定
- 有哪些現有分析結果

然後 AI 可以透過 FrameLab MCP 讀取上下文並回答。

0. 本輪最高原則

這一輪：

不要做：

- Generative Video 大整合
- Wan 大模型完整部署
- RIFE 完整生成工作流
- 多模型 Benchmark
- 大型訓練系統

- 複雜剪輯器
- 多軌音訊
- 字幕
- Premiere Clone
- After Effects Clone
- 多頁 Dashboard
- 社群功能
- 素材商城

本輪只做：

Frame Workspace + Conversation + Context + MCP

1. 最終使用情境

完成後，使用者可以：

1. 上傳一段影片
2. 系統拆成 Frames
3. Timeline 顯示逐幀
4. 點選 F135
5. 開 Onion Skin
6. 拖曳選取 F130–F140
7. 在 Canvas 圈選人物右手
8. 打開右側或浮動 AI 對話
9. 問：

這裡為什麼怪怪的？

系統建立：

```
{
  "project_id": "...",
  "timeline_id": "...",
  "current_frame": 135,
  "selected_range": {
    "start": 130,
    "end": 140
  },
  "selected_character": "...",
  "selected_region": {
    "type": "rectangle_or_mask",
    "frame_id": "...",
    "coordinates": {}
  },
  "onion_skin": {
```

```
"previous": 2,  
"next": 2  
}  
}
```

再透過 MCP:

```
get_current_context  
get_frame_neighbors  
get_selected_region  
analyze_selection
```

AI 最後能回答:

F135–F138 右手的位置變化和前後幀不連續。
目前選取區域在 F136 出現明顯位移。
建議進一步執行 motion / pose analysis。

第一版可以只分析，不要直接修改。

2. 先檢查現有 Repository

開始前:

```
git status  
git branch  
git log --oneline -20
```

閱讀目前:

- README
- architecture
- frontend
- backend
- MCP
- database
- timeline
- frame model

不要重做已經完成的功能。

不要覆蓋現有工作。

如果目前在 main:

建立:

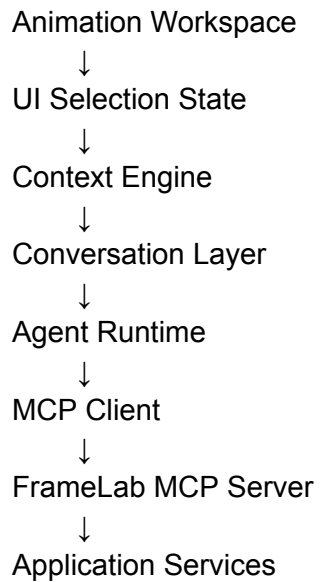
feat/context-conversation-core

3. 本輪真正核心

新增三個核心模組：

Context Engine
Conversation Layer
MCP Context Bridge

關係：



4. Context Engine

建立獨立模組，例如：

packages/context-engine/

不要把 context state 全部塞在 React Component 裡。

Context Engine 必須是 Domain/Application level。

至少管理：

project_id
video_id
timeline_id

current_frame_id
current_frame_number

selected_frame_ids
selected_frame_range

selected_character_id
selected_object_id

selected_region

onion_skin_settings

active_overlay

viewport

analysis_results

conversation_id

5. Current Context Schema

建立強型別 Schema。

例如：

```
interface FrameLabContext {  
  projectId: string | null  
  videoId: string | null  
  timelineId: string | null  
  
  currentFrame: {  
    id: string  
    frameNumber: number  
    timestampMs: number  
  } | null  
  
  selectedRange: {  
    startFrame: number  
    endFrame: number  
  } | null  
  
  selectedFrames: number[]  
  
  selectedCharacterId: string | null
```

selectedObjectId: string | null

selectedRegion: RegionSelection | null

```
onionSkin: {  
  enabled: boolean  
  previousFrames: number  
  nextFrames: number  
  previousOpacity: number  
  nextOpacity: number  
}
```

```
overlay: {  
  pose: boolean  
  mask: boolean  
  tracking: boolean  
  motion: boolean  
  depth: boolean  
  consistency: boolean  
}  
}
```

可以合理調整，但請維持清楚、可擴充。

6. Context 必須可以被序列化

AI / MCP 不應直接接 React State。

建立：

ContextSerializer

輸出：

```
{  
  "current_frame": 135,  
  "selected_range": [130, 140],  
  "selected_character": "character_xxx",  
  "selected_region": {},  
  "neighbors_available": true,  
  "analysis_available": []  
}
```

注意：

不要把整個 Project 所有 Frame 全塞給 LLM。

只傳必要內容。

7. Context Resolver

建立：

ContextResolver

例如：

使用者沒有明講 Frame：

「這裡為什麼怪？」

Context Resolver 必須知道：

current_frame = 135
selected_region = right hand selection
selected_range = 130–140

所以 AI 不需要使用者每次重複描述。

8. Context Priority

定義清楚優先順序：

Selected Region

↓

Selected Frame Range

↓

Current Frame

↓

Current Timeline

↓

Current Project

越局部的 Context 優先。

例如：

如果選了 F130–140, 又圈了 F135 的右手：

AI 預設關注：

F135 selected region
+
F130–140 temporal context

9. Frame Selection

Timeline 必須支援：

- single frame select
- multi frame select
- frame range select

可以：

click
shift-click
drag range

手機 / 平板可以先合理簡化。

Desktop 第一版完整。

10. Region Selection

Canvas 加入：

Region Selection Mode

第一版至少支援：

rectangle selection

如果現有 segmentation 架構可用，再支援 mask。

Region Schema：

frame_id
x
y
width
height
normalized_coordinates
selection_type

Coordinates 建議保存 normalized 0–1。

例如：

```
{  
  "x": 0.42,  
  "y": 0.31,  
  "width": 0.18,  
  "height": 0.24  
}
```

避免解析度改變造成選區失效。

11. Region Selection UX

使用者在 Canvas：

拖曳矩形。

Canvas 顯示選取框。

右側 Inspector 顯示：

Selected Region

Frame: F135

X: ...

Y: ...

W: ...

H: ...

提供：

Clear Selection

12. Onion Skin

這一輪 Onion Skin 必須真的可用。

UI：

Onion Skin

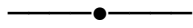
Previous

[1] [2] [3]

Next

[1] [2] [3]

Previous Opacity



Next Opacity



第一版預設：

previous = 2

next = 2

Canvas 需要顯示：

Previous Frames

Current Frame

Next Frames

要注意效能。

不要每次重新下載所有圖。

建立 preload/cache。

13. Frame Navigation

Keyboard Shortcuts:

至少：

Left Arrow = previous frame

Right Arrow = next frame

Space = play / pause

K = mark keyframe

O = onion skin toggle

Esc = clear region selection

如果快捷鍵和瀏覽器行為衝突，合理處理。

14. Floating AI Conversation

新增：

AI Conversation Overlay

它不是另外一頁。

它必須疊在 Animation Workspace 裡。

支援：

open

close

minimize

resize optional

drag optional

第一版優先：

- fixed right panel

或

- floating bottom-right panel

如果 Drag 會讓實作複雜，可以第二順位。

但不能跳離 Timeline。

15. Conversation Panel

至少：

Header

|— AI Provider

|— Mode

|— Context Status

Messages

Context Chips

Input

Send

例如：

Frame F135
Range F130–F140
Character Xiaohua
Region Selected

Context Chips 是 read-only visual indicator。

16. Conversation Modes

第一版做：

ASK

只讀。

不要讓 AI 自動修改 Frames。

架構預留：

ASSIST
AGENT

未來：

ASK
= read / analyze

ASSIST
= recommend action, human confirms

AGENT
= permitted autonomous operations

本輪只正式啟用 ASK。

17. Conversation Database

建立：

conversations
messages

至少：

conversations

- id
- project_id
- timeline_id
- title
- provider
- mode
- created_at
- updated_at

messages

- id
- conversation_id
- role
- content
- context_snapshot
- created_at

Context Snapshot 很重要。

每次 User message 都保存當時 Context。

18. Context Snapshot

例如使用者說：

這裡怪怪的

DB 不可以只保存文字。

同時保存：

```
{  
  "current_frame": 135,  
  "selected_range": [130, 140],  
  "selected_region": {},  
  "character_id": "..."  
}
```

這樣未來回看對話才知道「這裡」是什麼。

19. Conversation Thread Attached to Frame

第一版先讓 Conversation 可以關聯：

project
timeline
frame optional
frame_range optional

Frame Inspector 顯示：

Conversations

如果這張 Frame 有對話：

顯示：

 2

Timeline 也可以用小圖示表示有 conversation。

不要讓 UI 太花。

20. AI Provider Abstraction

建立：

LLMProvider

不要把 Grok API 寫死在 UI。

介面例如：

```
class LLMProvider:
    async def chat(
        messages,
        context,
        tools
    ):
        ...
```

支援 Adapter：

Grok
OpenAI
Claude
Gemini
Local

本輪至少實作一個真實 Provider, 如果環境有 Credentials。

沒有 Credentials:

Provider Status 正確顯示:

NOT_CONFIGURED

禁止假回覆。

21. Provider Registry

建立:

ProviderRegistry

可以查:

name

status

model

supports_tools

supports_vision

configured

Frontend 顯示 Provider Selector。

22. Conversation 不直接打資料庫

Conversation Runtime 只能透過:

Application Services

MCP

Context Engine

取得 Frame 資料。

不要讓 Prompt Builder 自己直接 SQL query。

23. MCP Context Tools

新增以下 MCP Tools:

get_current_context

get_current_frame

get_selected_frames

get_selected_frame_range

get_selected_region

get_frame_neighbors

get_current_character

get_current_object

24. get_current_context

Input:

session_id

Output:

```
{
  "project": {},
  "timeline": {},
  "current_frame": {},
  "selected_range": {},
  "selected_region": {},
  "onion_skin": {},
  "available_analysis": []
}
```

注意 Session Isolation。

不同使用者 / Browser Session 不能互相讀 Context。

25. get_frame_neighbors

Input:

frame_id
before
after

例如：

frame_id = F135
before = 2
after = 2

Output:

F133
F134
F135
F136
F137

只返回 metadata / asset refs。

不要把高解析度 binary 直接放 JSON。

26. get_selected_region

回傳：

frame
coordinates
preview asset reference
character association optional
object association optional

27. MCP Analyze Tools

本輪至少做：

analyze_selection

compare_frames

analyze_motion_context

如果 Pose / Flow Provider 還沒裝好：

清楚 fallback。

28. analyze_selection

Input:

context_id
analysis_types

例如：

```
{  
  "analysis_types": [  
    "visual",  
    "motion"  
  ]  
}
```

輸出：

```
{  
  "summary": "...",  
  "frames": [135,136,137],  
  "observations": [],  
  "available_metrics": {},  
  "limitations": []  
}
```

禁止產生不存在的 metric。

29. compare_frames

Input:

frame_a
frame_b
region optional

第一版可以使用：

- image diff
- perceptual similarity
- pixel displacement
- existing optical flow

如果模型不存在：

使用可解釋的 lightweight compare。

30. Lightweight Analysis

為了讓 V0.1 不被大型模型阻塞，實作：

OpenCV based lightweight analysis

可以有：

pixel difference
SSIM / perceptual-like metric
edge difference
histogram change
bounding region change

但請清楚標記這是：

lightweight visual analysis

不要把它包裝成 AI Pose。

31. VLM Integration

如果 Qwen-VL 或其他 Vision Provider 已配置：

可以讓 selected frame / region 做 VLM 分析。

不要：

240 Frames 全跑。

預設：

current frame
+
selected region
+
1-2 neighbors

32. AI Prompt Builder

建立：

ConversationContextBuilder

產生 prompt 時，分成：

System Instructions

User Message

FrameLab Context

Available Tools

Analysis Results

不要把所有資訊混成一大段無結構文字。

33. Prompt Context Example

CURRENT PROJECT

Project ID: ...

CURRENT TIMELINE

FPS: 24

CURRENT FRAME

Frame: 135

Timestamp: 5.625s

SELECTED RANGE

130–140

SELECTED REGION

normalized x/y/w/h

ONION SKIN

Previous: 2

Next: 2

AVAILABLE ANALYSIS

Lightweight visual diff

USER MESSAGE

「這裡為什麼怪怪的？」

34. AI 回覆要 Grounded

如果 FrameLab 沒有 Pose Analysis:

AI 不可以說:

右手腕 rotation = 32°

除非真的有資料。

應說:

目前只有視覺變化分析, 還沒有骨架資料。
從畫面差異看, F135–F137 選取區域存在明顯位置跳動。

35. Context Inspector

為 Debug 與開發, 建立:

Context Inspector

可以看到目前:

Current Frame
Selected Range
Selected Region
Character
Object
Onion Skin
Conversation

開發環境可完整。

Production 可以簡化或放 Debug Mode。

36. Timeline ↔ Context Engine

Timeline 每次:

seek
select
range select

都必須同步 Context。

不要讓：

Timeline state

Conversation state

Canvas state

各自有不同 current frame。

必須 single source of truth。

37. Canvas ↔ Context Engine

Region selection：

Canvas

↓

Region Selection

↓

Context Engine

Conversation 自動獲得新 Context。

38. Conversation Context Lock

加入可選：

Lock Context

例如：

使用者鎖定：

F130–140

即使之後點到 F150, 這個 Conversation Thread 還是分析原本 Range。

否則預設：

Follow Workspace Context

39. UI Context Indicator

AI Panel 上方顯示：

Following workspace

或：

Context locked
F130–F140

避免使用者不知道 AI 現在看哪裡。

40. Agent Runtime

本輪先建立簡單：

Conversation Runtime

流程：

User Message

↓

Resolve Context

↓

Build Prompt

↓

Determine MCP Tools

↓

Call AI Provider

↓

Tool Calls optional

↓

Response

↓

Persist Message

不要一開始建立超複雜 multi-agent orchestration。

41. Tool Call Logging

每一次 AI 呼叫 MCP Tool：

記錄：

conversation_id
message_id
tool
arguments
status
duration
result_summary

不要把大型 binary 記進 log。

42. Read-only Permission

本輪 Conversation AI 必須只有：

READ
ANALYZE

禁止：

EDIT
GENERATE
RENDER

即使 MCP Server 已經存在這些 Scope。

43. AI Action Suggestions

AI 可以回傳 Suggestion：

```
{  
  "type": "suggestion",  
  "action": "run_motion_analysis",  
  "frame_range": [135,138]  
}
```

UI 顯示：

建議：分析 F135–F138 動作
[執行分析]

但是使用者要按。

這不是自動 Edit。

44. Suggested Action Schema

建立：

SuggestedAction

例如：

ANALYZE_POSE
ANALYZE_MOTION
ANALYZE_TRACKING
COMPARE_FRAMES
MARK_PROBLEM

本輪不要開 Repair。

45. Conversation UX

不要做得像普通客服聊天室。

AI 是動畫助手。

建議 UI：

AI Assistant

Context:

F135

F130-140

Region selected

你：

這裡為什麼怪？

AI：

F135-137 的選取區域變化明顯高於前後幀。

目前沒有 Pose 資料，我建議跑 Motion Analysis。

[分析 Motion]

46. Error UX

Provider 不可用：

AI provider is not configured.

MCP 失敗：

Frame context could not be loaded.

Frame 不存在：

Selected frame no longer exists.

Job running：

Analysis running...

不要讓 UI 卡死。

47. Loading State

Conversation Tool Call：

顯示：

Reading frame context...

Analyzing selected frames...

Comparing neighboring frames...

不要假裝 instant。

48. Performance

不要每次使用者輸入一個字就重新 fetch Frames。

Context Engine 需要：

memoization

frame metadata cache

thumbnail cache

neighbor cache

49. Image Handling

傳給 Vision AI:

使用:

current frame
selected crop
neighbor thumbnails

不要無限制傳原始 4K Frames。

建立:

VisionAssetBuilder

自動產生適合 AI 分析的 preview。

50. Region Crop

如果 selected region:

建立 crop:

selected region
+
padding 10–20%

同時保留完整 Frame thumbnail 作 context。

51. Database Schema Update

新增至少:

conversations
conversation_messages
conversation_tool_calls
workspace_sessions
context_snapshots
region_selections

如果現有 schema 已能涵蓋, 避免重複 table。

52. Workspace Session

建立：

workspace_session

包含：

id
project_id
timeline_id
user optional
current_frame
selected_range
selected_region
context_version
updated_at

MCP 使用 session_id 取得當下 Context。

53. Context Version

每次重要 Context 改變：

context_version += 1

避免 AI Tool Call 使用過期 Context。

54. Stale Context

如果 AI 分析開始時：

context_version = 14

回傳時 Workspace 已經：

context_version = 20

UI 要能標示：

This answer refers to an earlier selection.

不要偷偷套用到新 Frame。

55. MCP Context Resource

新增:

framelab://sessions/{session_id}/context

可以返回 Context Snapshot。

56. MCP Conversation Resource

新增:

framelab://conversations/{conversation_id}

返回:

- message history
- linked frames
- linked ranges
- context snapshots

57. Timeline Conversation Marker

如果 Frame 有 Conversation:

Timeline 顯示小型:



如果 Frame 有 Analysis Warning:



如果兩者都有, 不要讓 UI 太擠。

58. Frame Inspector Conversation Section

顯示:

AI Conversations

- F135 hand movement
- Range F130–140

點擊可重新打開 Thread。

59. Testing

本輪必須新增：

Context Unit Tests

current frame update
range selection
region selection
context serialization
context priority
context lock
context version

Conversation Tests

create conversation
persist message
context snapshot
provider unavailable
tool call
stale context

MCP Tests

get_current_context
get_current_frame
get_selected_range
get_selected_region
get_frame_neighbors
permission denied
invalid session

UI Tests

至少：

timeline selects frame
canvas selection updates context

AI panel sees context
conversation survives navigation

60. E2E Smoke Test

建立真正 smoke flow:

Upload 2–5 sec video
↓
Extract frames
↓
Open timeline
↓
Select F10
↓
Enable onion skin
↓
Select F8–F12
↓
Draw region
↓
Open AI panel
↓
Ask "What looks inconsistent here?"
↓
Context Engine builds context
↓
MCP reads frames
↓
AI / fallback analyzer answers

如果 AI Provider 沒有 key:

仍應測:

Context
MCP
Lightweight Analysis

61. Definition of Done

這一輪完成標準:

1. Video import works

2. Frame extraction works
3. Timeline works
4. Frame navigation works
5. Frame range selection works
6. Onion Skin works
7. Region selection works
8. Context Engine exists
9. Context is single source of truth
10. Context can serialize
11. Workspace session exists
12. Context version exists
13. Floating / side AI panel exists
14. Conversation DB exists
15. Messages save Context Snapshot
16. MCP get_current_context works
17. MCP get_current_frame works
18. MCP get_selected_frame_range works
19. MCP get_selected_region works
20. MCP get_frame_neighbors works
21. analyze_selection works at least lightweight
22. AI Provider abstraction exists
23. At least one real provider can plug in
24. Missing provider handled correctly
25. Conversation AI is READ + ANALYZE only
26. Timeline can show conversation marker
27. Frame Inspector can show conversations
28. Tests pass
29. Frontend build passes
30. Docker still works

62. 不可以假裝完成

不要：

fake AI result
random motion score
random consistency
hardcoded assistant response
fake frame context
mock MCP success in production

Development fixture 可以存在。

Production path 不可 fake。

63. 不要破壞原本架構

如果 FrameLab 已經有：

Frame Graph
Timeline
MCP
REST API
Storage
Job System

請擴充現有 abstraction。

不要另外建立第二套。

64. 不要讓前端直接控制 MCP

正確：

Frontend
↓
Application API
↓
Conversation Runtime
↓
MCP Client
↓
MCP Server

避免 Browser 直接拿 MCP Admin Token。

65. Security

Context / Conversation 必須 Project Isolated。

MCP Token 只能讀允許 Project。

Region asset URL 不能任意讀取其他專案資源。

Conversation 不得暴露：

filesystem path
secret

API key
database credentials

66. Grok Provider

如果環境中有 xAI / Grok API Credentials:

可以建立:

GrokProvider

但是 Grok 只是 Provider。

不要:

FrameLab = Grok only

FrameLab 必須維持 Provider-agnostic。

67. 最重要的 Context Philosophy

FrameLab 的 AI 最大優勢不是:

chatbot

而是:

AI knows what the animator is currently looking at.

也就是:

Spatial Context

哪個區域。

+

Temporal Context

前後哪些 Frame。

+

Semantic Context

哪個角色 / 物件。

+

Conversation Context

之前討論過什麼。

這四個必須是 FrameLab Conversation Layer 的基礎。

68. 未來預留

本輪完成之後，下一階段才能加入：

ASSIST Mode

↓

Suggested Repair

↓

User Confirm

↓

Revision

↓

Repair Job

再下一階段：

AGENT Mode

↓

Analyze

↓

Repair

↓

Re-analyze

↓

Before / After

本輪禁止偷跑到完全自動修改。

69. 最終 UX

完成本輪後，FrameLab 應該讓人感覺：

我不是在跟一個不知道畫面的聊天機器人講話。

我是在跟一個正在跟我一起看同一個動畫 Timeline 的 AI 動畫助手對話。

這是這一輪最重要的產品體驗。

70. 執行順序

按照以下順序做：

1. Audit existing architecture
2. Context Engine
3. Workspace Session
4. Timeline selection integration
5. Region selection
6. Onion Skin stabilization
7. Conversation database
8. Conversation UI
9. MCP Context Tools
10. Lightweight Analysis
11. AI Provider abstraction
12. Conversation Runtime
13. Context Snapshot
14. Conversation markers
15. Tests
16. Documentation
17. Final audit

71. 文件

更新 / 建立：

docs/CONTEXT_ENGINE.md

docs/CONVERSATION_LAYER.md

docs/MCP.md

docs/AI_PROVIDERS.md

docs/ANIMATION_WORKSPACE.md

並更新：

README.md

ARCHITECTURE.md

72. Git 規則

不要使用：

git add .

git add -A

只 stage 本輪相關檔案。

檢查：

git diff

git diff --staged

不要：

- force push main

- reset 使用者工作

- 刪除未知檔案

- 亂改 unrelated modules

73. 最終回報格式

完成後回報：

Completed

本輪真正完成什麼。

Core Loop

目前：

Video

→ Frames

→ Timeline

→ Selection

→ Context

→ Conversation

→ MCP

→ Analysis

哪幾段真的打通。

Context Engine

Context 包含什麼。

Conversation

目前支援什麼。

MCP

新增哪些：

Resources

Tools

Prompts

AI Providers

哪些真的配置成功。

哪些未配置。

Tests

列實際測試結果。

UI

實際可以操作什麼。

Known Issues

不要隱藏問題。

Commands

如何：

啟動

測試

啟動 MCP

Next 5 Priorities

只給下一輪五項最高優先。

74. 最終最高原則

如果你需要在：

做出一個很漂亮但 AI 不知道使用者正在看什麼的聊天框

和：

做出一個外觀簡單，但 Context Engine 真正完全打通的 AI Conversation

之間選擇，

永遠選第二個。

本輪真正成功的標準是：

「使用者可以指著某一幀、某一段 Frame Range、某一塊畫面，直接跟 AI 說「這裡」，而 AI 真的知道「這裡」指的是什麼。」

現在開始實際開發。